



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

КБ-3 «Разработка программных решений и системное программирование»

Отчет по практической работе №7

**«Проект каталога: контейнеризация и подготовка к запуску»
по дисциплине**

«Сборка, тестирование и верификация программного продукта»

Выполнил студент группы БСБО-09-23

Фёдоров Антон Сергеевич

Работа выполнена

«21» мая 2026 г.

Москва, 2026 г.

СОДЕРЖАНИЕ

1. ЦЕЛЬ И ПЛАН РАБОТЫ	3
2. КОНТЕЙНЕРИЗАЦИЯ DJANGO-ПРОЕКТА.....	4
3. DOCKER COMPOSE, NGINX И VOLUMES.....	6
4. ТЕСТ-ПЛАН И МАТРИЦА ТЕСТ-КЕЙСОВ.....	8
5. РЕЗУЛЬТАТЫ ПРОВЕРКИ.....	11
6. ЧЕК-ЛИСТ ПО ТРЕБОВАНИЯМ	13
7. ИСХОДНЫЙ КОД И КОНФИГУРАЦИЯ	14
7.1. Dockerfile	14
7.2. Docker Compose	15
7.3. nginx	16
7.4. Entrypoint и зависимости	17
7.5. Настройки Django	18
ВЫВОДЫ	21

1. ЦЕЛЬ И ПЛАН РАБОТЫ

Цель: подготовить проект каталога аккумуляторов к воспроизводимому запуску вне машины разработчика: собрать Docker-образ Django-приложения, запустить проект через WSGI-сервер gunicorn, настроить nginx как внешний HTTP-прокси, вынести базу данных и статику во внешние Docker volumes, а также подтвердить качество линтером и тестами в контейнере.

План:

1. Изучить требования к практической работе 7.
2. Использовать проект каталога аккумуляторов из ПЗ6 как основу.
3. Подготовить Dockerfile для Django-приложения.
4. Добавить gunicorn как WSGI-сервер.
5. Настроить docker-compose.yml для сервисов web и nginx.
6. Настроить nginx как reverse проху к Django-контейнеру.
7. Вынести SQLite-базу в volume catalog_db.
8. Вынести собранную статику в volume static_files.
9. Добавить entrypoint для миграций, collectstatic и загрузки фикстур.
10. Настроить pylint и проверить код линтером.
11. Запустить Django-тесты в контейнере и оформить отчет.

2. КОНТЕЙНЕРИЗАЦИЯ DJANGO-ПРОЕКТА

Практическая работа 7 называется «Проект каталог. Подготовка к бою». Ее смысл - показать, что проект каталога работает не только локально, но и в воспроизводимом Docker-окружении.

Проект находится в папке ПЗ7/catalog_project. Бизнес-функциональность сохранена из ПЗ6: каталог аккумуляторов, роли пользователей, формы управления товарами, партии на отправку, оптовые цены и расчет итоговой суммы партии.

Инфраструктурный слой ПЗ7 состоит из следующих файлов:

- Dockerfile - сборка образа Django-приложения;
- docker-compose.yml - запуск связки web + nginx;
- requirements.txt - фиксированные зависимости проекта;
- scripts/entrypoint.sh - миграции, сбор статики и загрузка фикстур перед стартом приложения;
- nginx/default.conf - проксирование HTTP-запросов в Django-контейнер;
- setup.cfg - настройки pycodestyle;
- .dockerignore - исключение лишних файлов из Docker build context.

Docker-образ собирается из python:3.12-slim. Внутри создается рабочая директория /app, устанавливаются зависимости из requirements.txt, копируется код проекта и настраивается запуск от отдельного системного пользователя django. Это уменьшает риск запуска приложения от root внутри контейнера.

Контейнер Django запускает команду:

```
gunicorn catalog_project.wsgi:application --bind 0.0.0.0:8000 --workers 3
```

Таким образом, Django обслуживается через WSGI-сервер gunicorn, а не через development-сервер runserver.

3. DOCKER COMPOSE, NGINX И VOLUMES

В `docker-compose.yml` описаны два сервиса: `web` и `nginx`. Сервис `web` собирается из текущего `Dockerfile` и запускает Django-приложение. Сервис `nginx` использует образ `nginx:1.27-alpine`, принимает запросы с хоста на порту 8087 и проксирует их в `web:8000`.

Сервис `web` получает настройки через переменные окружения:

- `DJANGO_DEBUG=False`;
- `DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1,web`;
- `DJANGO_DB_PATH=/data/db.sqlite3`;
- `DJANGO_STATIC_ROOT=/static`;
- `DJANGO_SECRET_KEY=pz7-local-docker-secret-key`.

В `settings.py` эти значения читаются через `os.environ`. Благодаря этому контейнерный запуск отключает `DEBUG`, настраивает `ALLOWED_HOSTS` и переносит SQLite-файл из каталога проекта в `/data/db.sqlite3`.

База данных вынесена во внешний Docker volume `catalog_db`, который подключается к `web` как `/data`. Поэтому файл базы данных сохраняется вне жизненного цикла контейнера. Статические файлы вынесены в volume `static_files`: Django собирает их в `/static`, а `nginx` подключает тот же volume в режиме `read-only` и отдает `/static/` через alias `/static/`.

Перед запуском `gunicorn` выполняется `scripts/entrypoint.sh`. Он применяет миграции, собирает статику и загружает фикстуру `battery_catalog`. Это позволяет поднять контейнер с готовой базой каталога без ручной настройки после старта.

После запуска `docker compose up -d` каталог доступен по адресу:

`http://127.0.0.1:8087/`

Остановка контейнеров выполняется командой:

`docker compose down`

4. ТЕСТ-ПЛАН И МАТРИЦА ТЕСТ-КЕЙСОВ

Для практической работы подготовлен файл TEST_PLAN.md.
Идентификатор тест-плана: ТР-АКВ-2026-07.

Тест-план проверяет не новую бизнес-логику каталога, а готовность проекта к воспроизводимому запуску в Docker-окружении. Проверяются Dockerfile, docker-compose.yml, requirements.txt, setup.cfg, scripts/entrypoint.sh, nginx/default.conf, сервисы web и nginx, volume catalog_db и volume static_files.

Не проверяются внешний production-деплой, HTTPS/TLS, production-секреты, PostgreSQL, нагрузочное тестирование, горизонтальное масштабирование, CI/CD, резервное копирование volume и monitoring/logging для production-систем.

Критерий прохождения: Docker-образ собирается, web и nginx запускаются, приложение отвечает через nginx со статусом HTTP 200, база создается как /data/db.sqlite3, статика собирается в /static, pycodestyle завершается с exit code 0, а все 46 Django-тестов проходят в контейнере.

Матрица тест-кейсов

- TC-01. docker compose build собирает образ pz7-catalog-web без ошибок.
- TC-02. docker compose up -d запускает сервисы web и nginx.
- TC-03. docker compose ps показывает контейнеры в состоянии Up.
- TC-04. http://127.0.0.1:8087/ отвечает HTTP 200 через nginx.
- TC-05. Dockerfile запускает gunicorn catalog_project.wsgi:application.
- TC-06. nginx/default.conf проксирует запросы в web:8000.
- TC-07. catalog_db:/data выносит SQLite-базу во внешний volume.
- TC-08. В контейнере web существует файл /data/db.sqlite3.
- TC-09. static_files:/static используется для собранной статистики.
- TC-10. nginx отдает /static/ через alias /static/.
- TC-11. entrypoint выполняет python manage.py migrate --noinput.

- TC-12. `entrypoint` выполняет `python manage.py collectstatic --noinput`.
- TC-13. `entrypoint` выполняет `python manage.py loaddata battery_catalog`.
- TC-14. `DJANGO_DEBUG=False` отключает `DEBUG` в контейнере.
- TC-15. `ALLOWED_HOSTS` берется из `DJANGO_ALLOWED_HOSTS`.
- TC-16. Линтер `pycodestyle` в контейнере завершается с `exit code 0`.
- TC-17. Django-тесты в контейнере проходят: 46 тестов, ОК.
- TC-18. `setup.cfg` задает `max-line-length = 120` и исключения.
- TC-19. `requirements.txt` содержит Django, gunicorn и `pycodestyle`.
- TC-20. Каталог открывается после запуска через `nginx`.
- TC-21. Карточка товара открывается после запуска через `nginx`.
- TC-22. Формы и роли из ПЗ6 не сломаны: `catalog.tests` проходят.
- TC-23. `docker compose down` останавливает контейнеры без ошибки.

5. РЕЗУЛЬТАТЫ ПРОВЕРКИ

Команда сборки образа:

```
docker compose build
```

Команда запуска приложения:

```
docker compose up -d
```

Проверка приложения через nginx:

```
Invoke-WebRequest -Uri 'http://127.0.0.1:8087/' -UseBasicParsing
```

Результат проверки: HTTP 200. Контейнеры web и nginx находятся в статусе Up.

Проверка базы данных во внешнем volume:

```
docker compose exec -T web sh -c "ls -l /data && test -f /data/db.sqlite3 && echo DB_FILE_OK"
```

Результат: файл /data/db.sqlite3 создан в volume catalog_project_catalog_db, выводится DB_FILE_OK.

Команда запуска линтера в контейнере:

```
docker compose run --rm --no-deps --entrypoint= web pycodestyle .
```

Фактический результат:

```
exit code: 0
pycodestyle status: OK
```

Команда запуска тестов в контейнере:

```
docker compose run --rm --no-deps --entrypoint= web python manage.py test catalog.tests --verbosity=1
```

Фактический результат:

```
Found 46 test(s).
Ran 46 tests.
OK
tests status: OK
```

Дополнительная локальная проверка: команда `docker compose config` выполняется успешно и показывает корректную конфигурацию сервисов web и nginx, volumes catalog_db и static_files. Локальная среда не заменяет контейнерную проверку, потому что на машине доступен Python 3.14 и Django 6.0.4, а Dockerfile использует Python 3.12 и зависимости из requirements.txt.

Рисунок 1. Успешный запуск pycodestyle в контейнере

PZ7: pycodestyle in Docker container

```
PS D:\Github\university-projects\037\catalog_project> docker compose run --rm --no-deps --entrypoint= web pycodestyle .  
exit code: 0  
pycodestyle status: OK
```

Рисунок 2. Успешный запуск Django-тестов в контейнере

PZ7: Django tests in Docker container

```
PS D:\Github\university-projects\037\catalog_project> docker compose run --rm --no-deps --entrypoint= web python manage.py test  
catalog.tests --verbosity=1  
Found 46 test(s).  
Creating test database for alias 'default'...  
System check identified no issues (0 silenced).  
.....  
-----  
Ran 46 tests in 5.070s  
  
OK  
Destroying test database for alias 'default'...  
exit code: 0  
tests status: OK
```

6. ЧЕК-ЛИСТ ПО ТРЕБОВАНИЯМ

- [x] Сделан Docker-образ Django-проекта.
- [x] Для запуска Django используется WSGI-сервер gunicorn.
- [x] База данных вынесена из контейнера во внешний Docker volume.
- [x] Через Docker Compose поднимаются два контейнера: web и nginx.
- [x] nginx настроен как внешний HTTP-прокси к Django-приложению.
- [x] Приложение доступно через `http://127.0.0.1:8087/`.
- [x] Настройки `DEBUG`, `ALLOWED_HOSTS`, `DB_PATH`, `STATIC_ROOT` и `SECRET_KEY` передаются через `environment`.
- [x] Статика собирается в отдельный volume `static_files`.
- [x] nginx получает `static_files` в режиме `read-only`.
- [x] `entrypoint` выполняет миграции.
- [x] `entrypoint` выполняет `collectstatic`.
- [x] `entrypoint` загружает фикстуру `battery_catalog`.
- [x] Добавлен `requirements.txt` с Django, gunicorn и pycodestyle.
- [x] Добавлен `setup.cfg` для pycodestyle.
- [x] Линтер проходит в контейнере со статусом ОК.
- [x] 46 Django-тестов проходят в контейнере со статусом ОК.
- [x] Приложены скриншоты успешного запуска линтера и тестов.

7. ИСХОДНЫЙ КОД И КОНФИГУРАЦИЯ

7.1. Dockerfile

Листинг 1. Файл Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

RUN addgroup --system django \
    && adduser --system --ingroup django django \
    && mkdir -p /data /static \
    && chown -R django:django /data /static /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY --chown=django:django . .
RUN chmod +x /app/scripts/entrypoint.sh

USER django

EXPOSE 8000

ENTRYPOINT ["/app/scripts/entrypoint.sh"]
CMD ["gunicorn", "catalog_project.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "3"]
```

7.2. Docker Compose

Листинг 2. Файл docker-compose.yml

```
services:
  web:
    build: .
    image: pz7-catalog-web
    restart: unless-stopped
    environment:
      DJANGO_DEBUG: "False"
      DJANGO_ALLOWED_HOSTS: "localhost,127.0.0.1,web"
      DJANGO_DB_PATH: "/data/db.sqlite3"
      DJANGO_STATIC_ROOT: "/static"
      DJANGO_SECRET_KEY: "pz7-local-docker-secret-key"
    volumes:
      - catalog_db:/data
      - static_files:/static

  nginx:
    image: nginx:1.27-alpine
    restart: unless-stopped
    depends_on:
      - web
    ports:
      - "8087:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
      - static_files:/static:ro

volumes:
  catalog_db:
  static_files:
```

7.3. nginx

Листинг 3. Файл nginx/default.conf

```
upstream django_app {
```

```

    server web:8000;
}

server {
    listen 80;
    server_name localhost;

    client_max_body_size 10m;

    location /static/ {
        alias /static/;
        access_log off;
        expires 30d;
    }

    location / {
        proxy_pass http://django_app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

7.4. Entrypoint и зависимости

Листинг 4. Файл scripts/entrypoint.sh

```

#!/bin/sh
set -e

python manage.py migrate --noinput
python manage.py collectstatic --noinput
python manage.py loaddata battery_catalog

exec "$@"

```

Листинг 5. Файл requirements.txt

```

Django==4.2.11
gunicorn==23.0.0
pycodestyle==2.14.0

```

Листинг 6. Файл setup.cfg

```

[pycodestyle]
max-line-length = 120
exclude = */migrations/*,__pycache__,.venv,staticfiles

```

Листинг 7. Файл .dockerignore

```

.coverage
__pycache__/
*.py[cod]
db.sqlite3
.git/
.idea/
screenshots/

```

7.5. Настройки Django

Листинг 8. Файл catalog_project/settings.py

```

"""
Django settings for catalog_project project.

Generated by 'django-admin startproject' using Django 6.0.4.

```

```

For more information on this file, see
https://docs.djangoproject.com/en/6.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/6.0/ref/settings/
"""

import os
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = os.environ.get(
    'DJANGO_SECRET_KEY',
    'django-insecure--2^@gze6y7naa5j^6ifb3m^m8(81(jl+)1f)hy231tto)3yuje',
)

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = os.environ.get('DJANGO_DEBUG', 'True').lower() == 'true'

ALLOWED_HOSTS = os.environ.get('DJANGO_ALLOWED_HOSTS', 'localhost,127.0.0.1').split(',')

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'catalog',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'catalog_project.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'catalog_project.wsgi.application'

# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': os.environ.get('DJANGO_DB_PATH', BASE_DIR / 'db.sqlite3'),
    }
}

```

```

}

# Password validation
# https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/6.0/topics/i18n/

LANGUAGE_CODE = 'ru-ru'

TIME_ZONE = 'Europe/Moscow'

USE_I18N = True

USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/6.0/howto/static-files/

STATIC_URL = '/static/'
STATIC_ROOT = os.environ.get('DJANGO_STATIC_ROOT', BASE_DIR / 'staticfiles')

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

```

ВЫВОДЫ

В результате выполнения практической работы проект каталога аккумуляторов был подготовлен к запуску в воспроизводимом Docker-окружении.

Для Django-приложения создан Dockerfile на базе python:3.12-slim. Приложение запускается через WSGI-сервер gunicorn. Через docker-compose.yml поднимаются два сервиса: web с Django-приложением и nginx, который принимает HTTP-запросы с хоста и проксирует их в web.

База данных SQLite вынесена во внешний volume catalog_db, а собранная статика - во внешний volume static_files. При старте контейнера entrypoint применяет миграции, собирает статику и загружает фикстуру каталога.

Качество проекта подтверждено контейнерными проверками: pytest завершился со статусом OK, а 46 Django-тестов прошли без ошибок. Дополнительно локально проверена корректность docker compose config.

Таким образом, цель практической работы достигнута.